THE ZERO PARADOX

ZP-J AFA Addendum

Decoration Uniqueness from Valuation Structure

Initial: May 2026 | Current: 1.19, October 2026

This document presents the formal consequences of ZP-J's valuation framework for the central uniqueness CLAUSE of Aczel's Anti-Foundation Axiom (AFA). The main result is decoration_unique (ZeroParadox/Settheory/APG.lean § IX): for any finite Accessible Pointed Graph, any two valid decorations must agree at every vertex. The proof does not import set-theoretic AFA axioms. It derives the uniqueness property from a chain of abstract typeclasses whose root is ValuationStructure, established in ZP-J.

Readers of ZP-J Self-Reference will recognise the key move: ⊥ is the unique fixed point of scale because any non-⊥ element has finite depth, and scale increases depth by 1. Depth is what settles the cyclic case too, though by bounding rather than by iterating: going once around a cycle forces a vertex's depth to exceed itself, which only infinite depth — that is, ⊥ — can do. Acyclic vertices are handled by strong induction on the size of the reachable set. Both cases together give decoration_unique.

Section I: ValuationStructure and its Unique Fixed Point

ValuationStructure is the root typeclass of the derivation chain. It abstracts the depth-measure argument that drives every uniqueness result in this document. A type L carries a ValuationStructure when it has a ZPSemilattice structure, a self-application operation scale, and a depth measure val taking values in ℕ∞ (the natural numbers extended with a point at infinity).

Typeclass: ValuationStructure (ZeroParadox/Valuation/Scale.lean)
class ValuationStructure (L : Type*) [ZPSemilattice L] where
scale : L → L
val : L → ℕ∞
scale_bot : scale bot = bot
val_bot : val bot = ⊤
val_unique : ∀ x : L, val x = ⊤ → x = bot
val_scale : ∀ x : L, x ≠ bot → val (scale x) = val x + 1

Typeclass: ValuationStructure (ZeroParadox/Valuation/Scale.lean)

Transcribed from ZeroParadox/Valuation/Scale.lean. ⊤ is the top of ℕ∞, written ∞ in the prose above; bot is ZPSemilattice's ⊥. Read: scale fixes the bottom, the bottom has infinite depth, only the bottom does, and scale adds one everywhere else.

val_unique and val_scale carry this section's argument: val_unique gives, for any x ≠ ⊥, that val(x) is finite — it is the contrapositive of "infinite depth identifies ⊥" — and val_scale makes scale strictly increase it. No element of finite depth can therefore satisfy scale(x) = x. val_bot is consumed nowhere on that chain (measured 2026-08-30); it fixes the depth of ⊥ itself, which the fixed-point argument never reads.

Theorem: scale_unique_fp (ZeroParadox/Valuation/Scale.lean)

∀ x : L, ValuationStructure.scale x = x → x = bot

⊥ is the only fixed point of scale.

Proof: suppose scale x = x and x ≠ ⊥. By val_scale, val(scale x) = val(x) + 1. But scale x = x gives val(x) = val(x) + 1, which has no finite solution in ℕ∞, and val(x) is finite by val_unique. So scale x ≠ x for every x ≠ ⊥ (scale_ne_fixed), a contradiction.

Lean purity: [propext, Classical.choice, Quot.sound]. ✓

Section II: The Derivation Chain

ValuationStructure generates two further typeclasses by successive derivation. AbstractSelfApp extracts the fixed-point structure from ValuationStructure, replacing scale with an abstract selfApp operation. AFAStructure — the three-field typeclass encoding the Quine atom properties — is then derived from AbstractSelfApp. At each step the LAWS of the target typeclass are proved as theorems from the source, and the DATA fields are supplied by definition: the selfApp field of AbstractSelfApp is assigned (selfApp := scale), and the selfMem field of AFAStructure is supplied by def selfMemDerived. TWO of the three AFAStructure fields become theorems, not three (ZeroParadox/Computability/SelfApp.lean, def selfMemDerived). No new axioms are introduced. The relationship to Aczel's Anti-Foundation Axiom in ZF+AFA is discussed in Remark R-J.A (§V).

Typeclass: AbstractSelfApp (ZeroParadox/Computability/SelfApp.lean)

class AbstractSelfApp (L : Type*) [ZPSemilattice L] where

selfApp : L → L

fixed_bot : selfApp bot = bot

unique_fp : ∀ x : L, selfApp x = x → x = bot

Instance toAbstractSelfApp (ZeroParadox/Valuation/Scale.lean):

selfApp := scale

Typeclass: AbstractSelfApp (ZeroParadox/Computability/SelfApp.lean)
fixed_bot := scale_bot (direct from ValuationStructure)
unique_fp := scale_unique_fp (proved in §I above)
No new axioms.

AFAStructure is the lattice-level encoding of the three structural facts that ZF+AFA provides set-theoretically: that ⊥ contains itself, that it is the only self-containing element, and the self-membership predicate itself. In ZF+AFA, the existence of a self-containing set follows from AFA's existence clause applied to the one-node self-loop graph; uniqueness follows from AFA's uniqueness clause. In the ZP encoding, the existence field (bot_self_mem) is a LAW, discharged by a theorem built from fixed_bot in AbstractSelfApp. The uniqueness field (quine_unique) is likewise a law, proved from unique_fp. Only selfMem is data, and only it is supplied — no new axioms are introduced at this step.

Typeclass: AFAStructure (ZeroParadox/Settheory/SetTheoryAFA.lean)
class AFAStructure (L : Type*) [ZPSemilattice L] where
selfMem : L → Prop
quine_unique : ∀ x y : L, selfMem x → selfMem y → x = y
bot_self_mem : selfMem bot
Instance toAFAStructure (ZeroParadox/Computability/SelfApp.lean):
selfMem := selfMemDerived (DATA - supplied, a def)
bot_self_mem := derived_bot_self_mem (LAW - discharged by a theorem)
quine_unique := derived_quine_unique (LAW - discharged by a theorem)
No new axioms. Two of the three fields are laws and become theorems;
selfMem is data and is supplied.

Proposition: Derivation Chain (ZeroParadox/Valuation/Scale.lean, ZeroParadox/Computability/SelfApp.lean)
ValuationStructure L ⇒ AbstractSelfApp L ⇒ AFAStructure L
Each arrow is a Lean instance derivation proved without new axioms.
Any type satisfying ValuationStructure inherits the two AFAStructure LAWS as theorems; its data field selfMem is supplied by definition, not proved.
Lean purity: [propext, Classical.choice, Quot.sound]. ✓

Remark: Scope of the Chain

The derivation chain shows that ValuationStructure is sufficient to satisfy AFAStructure's three fields within the ZP typeclass hierarchy. It does not show that AFA is derivable from ZF: Foundation and AFA remain mutually exclusive set-theoretic frameworks. The chain is internal to the ZP lattice abstraction and says nothing about which set-theoretic axioms hold. The precise relationship to Aczel's Anti-Foundation Axiom is discussed in Remark R-J.A (§V).

Section III: Accessible Pointed Graphs and Decorations

An Accessible Pointed Graph (APG) is the combinatorial setting for the unique-decoration property AFA asserts — an AXIOM, not a theorem; see Remark R-J.A. The decoration uniqueness theorem asserts that any two valid labellings of an APG's vertices must agree. This section defines both notions in the abstract setting of ZP's DecorationUniverse typeclass.

Definition: Accessible Pointed Graph (ZeroParadox/Settheory/APG.lean §I)

An APG over vertex type V (a Quiver) is a structure APG V with:

root : V

accessible : ∀ v : V, Nonempty (Quiver.Path root v)

Every vertex is reachable from root by following directed edges.

children(v) := apg_children v = { w | Nonempty (v ⟶ w) } (immediate successors)

Reach(v) = { w : V | Nonempty (Quiver.Path v w) } (reachable from v)

In a finite APG (Fintype V), every Reach(v) is a finite set. |Reach(v)| is the cardinality used in the induction of ZeroParadox/Settheory/APG.lean § IX (this document's §IV.2).

A decoration assigns labels from a DecorationUniverse to each vertex, subject to a local consistency condition: the label at v is assembled from the labels of its immediate successors via the collect operation.

Typeclass: DecorationUniverse (ZeroParadox/Settheory/APG.lean §II)

class DecorationUniverse (U : Type*) [ZPSemilattice U] [ValuationStructure U] where

collect : Set U → U

collect_singleton : ∀ x : U, collect {x} = ValuationStructure.scale x

collect_val_ge : ∀ (S : Set U) (x : U), x ∈ S →

ValuationStructure.val (collect S) ≥ ValuationStructure.val x + 1

Typeclass: DecorationUniverse (ZeroParadox/Settheory/APG.lean §II)

Transcribed from ZeroParadox/Settheory/APG.lean. IsDecoration d means ∀ v, d v = collect (d '' apg_children v), where d '' S is Lean's set image — { d w | w ∈ S }. • The two axioms pin only the SINGLETON case and a lower bound; they do not require collect to assemble a parent's value from its children's.

Section IV: Decoration Uniqueness

The main theorem asserts that any finite APG admits at most one valid decoration. The proof splits on whether a vertex lies on a directed cycle.

Theorem: decoration_unique (ZeroParadox/Settheory/APG.lean §IX)

∀ {V : Type*} [Quiver V] [Fintype V]

{U : Type*} [ZPSemilattice U] [ValuationStructure U] [DecorationUniverse U]

(_G : APG V) (d_1 d_2 : V → U),

IsDecoration d_1 → IsDecoration d_2 → $d_1 = d_2$

For any finite APG, any two valid decorations into a DecorationUniverse agree at every vertex.

Lean purity: [propext, Classical.choice, Quot.sound]. ✓

IV.1 — Cyclic Vertices

A vertex v is cyclic if there exists a directed path from v back to itself. The valuation argument forces any valid decoration to assign ⊥ to every cyclic vertex, independent of which decoration is used. Both d_1 and d_2 must therefore assign ⊥ to every cyclic vertex, and they agree trivially on this case.

The argument is a chain of two inequalities, not an iterated equation. Let w be the next vertex on the cycle. Because w is a child of v and d(v) collects over d of v's children, collect_val_ge gives val(d(v)) ≥ val(d(w)) + 1. Following the cycle back from w to v, path_val_chain gives val(d(w)) ≥ val(d(v)) + length(p). Together these force val(d(v)) to exceed itself, which no finite value in ℕ∞ can do, so val(d(v)) = ∞ and therefore d(v) = ⊥.

Lemma: val_iterate (ZeroParadox/Settheory/APG.lean § III)

∀ {U : Type*} [ZPSemilattice U] [ValuationStructure U] (x : U),

x ≠ ZPSemilattice.bot → ∀ (k : ℕ),

ValuationStructure.val (ValuationStructure.scale^[k] x) =

ValuationStructure.val x + ↑k

For any x ≠ ⊥, applying scale k times increases depth by exactly k.

Lemma: val_iterate (ZeroParadox/Settheory/APG.lean § III)
Proof: induction on k; val_scale applies at each step because scale^[n](x) ≠ ⊥ follows from the induction hypothesis and finiteness of val(x).
• This is a genuine lemma of § III and it drives scale_iterate_unique_fp below. It is NOT what the cyclic case above uses: cyclic_decoration_eq_bot never forms scale^[k].
Lean purity: [propext, Classical.choice, Quot.sound]. ✓

Lemma: scale_iterate_unique_fp (ZeroParadox/Settheory/APG.lean §IV)
∀ {U : Type*} [ZPSemilattice U] [ValuationStructure U] (k : ℕ),
0 < k → ∀ (x : U), ValuationStructure.scale^[k] x = x →
x = ZPSemilattice.bot
⊥ is the only element fixed by any k-fold iteration of scale (k ≥ 1).
Proof: if scale^[k] x = x and x ≠ ⊥, then val_iterate gives val(x) = val(x) + k with k ≥ 1 — contradiction.
Lean purity: [propext, Classical.choice, Quot.sound]. ✓

Theorem: cyclic_decoration_eq_bot (ZeroParadox/Settheory/APG.lean § VII′)
∀ {V : Type*} [Quiver V] {U : Type*} [ZPSemilattice U]
[ValuationStructure U] [DecorationUniverse U] (d : V → U),
IsDecoration d → ∀ (v : V), HasSelfCycle v → d v = ZPSemilattice.bot
Consequence: d_1 v = d_2 v = ⊥ for every cyclic vertex v.
Proof route, as written: collect_val_ge across one edge gives val(d v) ≥ val(d w) + 1, path_val_chain around the cycle gives val(d w) ≥ val(d v) + length(p), and the two together are unsatisfiable for finite val. It consumes val_unique through val_finite_of_ne_bot, and assumes nothing about how many children a vertex has.
Lean purity: [propext, Classical.choice, Quot.sound]. ✓

IV.2 — Acyclic Vertices

A vertex v is acyclic if no directed cycle passes through it. The argument uses strong induction on |Reach(v)|. Every vertex reaches itself via the empty path, so |Reach(v)| ≥ 1 for every v; the n = 0 branch is vacuous. All actual vertices fall into the inductive step.

Inductive step. Suppose d_1 and d_2 agree on every vertex w with |Reach(w)| < n, and suppose |Reach(v)| = n with v acyclic. For each successor w ∈ children(v): since v is acyclic, v is not reachable from w, so Reach(w) ⊊ Reach(v), giving |Reach(w)| < n. By the induction hypothesis, d_1(w) = d_2(w) for every w ∈ children(v). When children(v) = ∅ this hypothesis is vacuously satisfied: collect is a function, the image of ∅ under any decoration is ∅, so both d_1(v) and d_2(v) equal collect(∅) and agree. When children(v) ≠ ∅, the induction

hypothesis gives d_1 '' children(v) = d_2 '' children(v), and the decoration equation then gives d_1(v) = collect(d_1 '' children(v)) = collect(d_2 '' children(v)) = d_2(v). In both sub-cases, acyclic_induction_step formalises the argument.

Lemma: acyclic_induction_step (ZeroParadox/Settheory/APG.lean §VIII)
If d_1 and d_2 agree on every successor of an acyclic vertex v,
then d_1 v = d_2 v.
Proof: collect is the same operation for both; d_1 and d_2 agree pointwise on children(v) by hypothesis; therefore the assembled labels agree.
Lean purity: [propext, Classical.choice, Quot.sound]. ✓

IV.3 — Combining the Cases

Every vertex in a finite APG is either cyclic or acyclic. The two cases are exhaustive and jointly establish d_1(v) = d_2(v) for every vertex v. Function extensionality then closes the global equality $d_1 = d_2$ from that pointwise agreement; finiteness is consumed earlier, by the acyclic case's descent on |Reach(v)|, and plays no part in this last step.

Section V: Scope, Purity, and Open Questions

decoration_unique establishes the uniqueness half of the property AFA asserts (an axiom — Remark R-J.A) for abstract DecorationUniverses over finite graphs. Two scope boundaries are worth stating explicitly.

Existence. This document does not prove that every finite APG admits a valid decoration. Uniqueness and existence are independent: one can show that no two decorations can differ without showing that any decoration exists. The existence half is not formalised here for abstract DecorationUniverses and remains an open question.

Finite graphs. decoration_unique requires Fintype V. The strong induction on |Reach(v)| terminates because V is finite. Extending the result to infinite APGs would require an ordinal induction or a well-foundedness argument on the reachability relation, and is not addressed here.

Commented-out stub. ZeroParadox/Settheory/APG.lean contains a commented-out theorem acyclic_decoration_unique with a sorry placeholder. That stub is not used by the proof of decoration_unique: the final proof proceeds by direct strong induction using acyclic_induction_step, without using the stub. All active (non-commented-out) theorems in the eight source files are sorry-free.

Lean Source Files
ZeroParadox/Valuation/Scale.lean — ValuationStructure, scale_unique_fp, toAbstractSelfApp
ZeroParadox/Computability/SelfApp.lean — AbstractSelfApp, selfMemDerived, derived_bot_self_mem, derived_quine_unique, toAFAStructure

Lean Source Files
ZeroParadox/Settheory/SetTheoryAFA.lean — AFAStructure typeclass (selfMem, quine_unique, bot_self_mem), IsQuineAtom
ZeroParadox/Settheory/AczelConn.lean — J_self, selfMem_determines_singleton, DC-free identification theorems
ZeroParadox/Settheory/OntBridge.lean — OntologicalStates as AbstractSelfApp instance
ZeroParadox/Settheory/Model.lean — instNatInfZPS and instNatInfVal, ℕ∞ as a ValuationStructure instance
ZeroParadox/Valuation/ScaleBridge.lean — instZ2ValBridge : ValBridge ℤ_[2]. Note no ValuationStructure ℤ_[2] is REGISTERED, nor any ZPSemilattice ℤ_[2] — which is a fact about what is declared, not about what is possible: ZeroParadox/Valuation/Scale.lean § V builds one and discharges all four axioms over it. ValBridge is the variant that drops the semilattice requirement while keeping the same four axioms.
ZeroParadox/Settheory/APG.lean — APG, DecorationUniverse, val_iterate, scale_iterate_unique_fp, cyclic_decoration_eq_bot, acyclic_induction_step, decoration_unique
All paths are repository-relative, in the public repository.

Axiom Footprint — measured per declaration, not asserted document-wide
[propext, Classical.choice, Quot.sound]
propext — propositional extensionality (standard in Lean 4)
Classical.choice — AT LEAST FOUR ROUTES, and they are not the same claim. (a) THE CLASSES: the numeral `1` in val_scale needs Mathlib's AddMonoidWithOne ℕ∞ instance, choice-tainted at the INSTANCE level, so ValuationStructure carries the axiom as a bare typeclass before any theorem; DecorationUniverse's own `+ 1` in collect_val_ge reaches the same instance. (b) A DIRECT CALL: decoration_unique's proof takes an edge out of a Nonempty via hw.some, and Nonempty.some IS Classical.choice — not inherited through packaging, invoked. (c) CARDINALITY: the termination measure runs on Set.ncard, and Set.ncard_lt_ncard and Set.ncard_pos are each independently choice-tainted. (d) THE CASE SPLIT: the proof's by_cases on HasSelfCycle u is decided by Classical.propDecidable, one of the three classical-instance gateways through which, jointly, 55.4% of Mathlib's choice-dependence flows (L. J. Close, The Axiom Profile of Mathlib, Zenodo 2026, doi:10.5281/zenodo.20710921, pp. 5–6; cited, not claimed).
Quot.sound — quotient soundness (standard in Lean 4)

Axiom Footprint — measured per declaration, not asserted document-wide

• A RESTATEMENT CLEARS IT FOR THE CLASS; NO PROOF CAN CLEAR IT FOR THE THEOREM AS STATED — and the difference is the whole claim. ZeroParadox/Valuation/Scale.lean transcribes the class twice, as _VSlit and _VScast, differing only in route (a)'s numeral, which _VScast writes ((1 : ℕ) : ℕ∞): _VSlit reports [propext, Classical.choice, Quot.sound] and _VScast reports no axioms. The declared class still reports the axiom; _VScast is a respelling of it, not a proof. decoration_unique's STATEMENT carries the axiom: a theorem that only assumes that statement and proves True reports [propext, Classical.choice, Quot.sound] (the statement control, measured 2026-10-09), so no proof of decoration_unique as stated can remove it — the STATEMENT-CARRIED class of ZeroParadox/Category/ChoiceCannotBe.md, recorded at DecorationUniverse in ZeroParadox/Settheory/APG.lean. Routes (b), (c) and (d) lie in the proof, not the statement. Measured 2026-10-09: the same statement restated over clean spellings of both classes reports only [propext, Quot.sound] under the statement control, and route (b)'s edge step, re-proved with obtain in place of .some, reports no axioms; routes (c) and (d) were not re-proved in that measurement.

No ZP-specific AXIOM is declared anywhere in this chain; the footprint above is what the Lean reports, not a commitment the framework makes.

No Dependent Choice. No additional set-theoretic assumptions.

Remark R-J.A — Relationship to Aczel's Anti-Foundation Axiom

Aczel's ANTI-FOUNDATION AXIOM (Non-Well-Founded Sets, CSLI 1988, ch. 1 p. 6) states that every GRAPH has a unique decoration into the universe of sets — an axiom, not a theorem, and quantified over graphs rather than over accessible pointed ones. (The book's decoration THEOREM is Mostowski's Collapsing Lemma, for WELL-FOUNDED graphs.) The result here is not a re-proof of Aczel's axiom by different methods. The objects are different: Aczel's target is a specific set-theoretic universe; the target here is any type carrying ValuationStructure and a collect operation, with no set-membership semantics required. The contribution is the generalisation: decoration uniqueness holds for any abstract DecorationUniverse satisfying the depth-measure axioms, independently of set-theoretic content. Whether the existence half of Aczel's axiom generalises to abstract DecorationUniverses is an open question.

Endnote: This document is an addendum to ZP-J Self-Reference and reads after it. The derivation chain (ValuationStructure → AbstractSelfApp → AFAStructure) is established in ZP-J; this document applies it to the APG decoration problem. It covers the uniqueness result for finite graphs. All active theorems are sorry-free in Lean 4 (one commented-out stub; see §V).